

Loss-function comparison: persistence/privilege_escalation collapse

All runs below share the same base config unless noted: host-fixed data (both HOST_PRIORITY fixes applied – videosever priority + <5-event sparse-primary fallback), stratified split (every tactic guaranteed ≥ 1 test example), $-\text{min-events } 3$, $\text{test-size } 0.2$.

NOTE: this file has disappeared from disk multiple times during this session (not from any edit made here) – if it goes missing again, check for an external cleanup process before assuming the data is gone; the underlying result JSONs in this same results/ directory are the source of truth and seem stable.

Background

Baseline run (#1 below) showed a severe collapse: the model defaults to predicting {persistence, privilege_escalation} as its top-2 regardless of the true label, on 25 of 39 test samples (original, pre-rebuild data). Verified concretely: of 17 times privilege_escalation was predicted #1, only 8 were actually correct (47% – worse than a coin flip). Root cause: persistence (40 train examples) and privilege_escalation (39) vastly outnumber rare tactics like exfiltration, impact, reconnaissance (2 each).

Tried two families of fix: uniformity_loss (label-blind, penalizes batch embeddings clustering together) and class-reweighting (label-aware, multiplies each tactic’s loss contribution by $\text{mean_count}/\text{count}$ so rare tactics count for more). `prototype_multilabel_loss.py` and `train_camlds_matcher.py` gained `--lambda-uniform` and `--class-reweight` [`--reweight-cap`] CLI flags for this.

Single-seed results (split-seed 1, template c, ORIGINAL pre-rebuild sequences)

#	Config	Best train loss	LRAP	AUPR	Wrong (top-3)	Persist+PrivEsc as top-1
1	Baseline (no uniform, no reweight)	1.419	62.3%	51.8%	13/39	17/39 (9 of 17 wrong)
2	Uniform loss = 0.1	1.030	61.8%	56.3%	11/39	~17/39 unchanged
3	Uniform loss = 0.5	-0.175	63.7%	52.9%	11/39	~17/39 unchanged
4	Class reweight, uncapped (max 8.21x)	–	30.0%	45.2%	30/39	0/39 (new collapse elsewhere)
5	Class reweight capped 3.0x + uniform 0.2	0.763	49.0%	49.0%	18/39	0/39 (softer new collapse elsewhere)

Multi-seed validation of config #2 (uniform loss = 0.1), 4 different splits, ORIGINAL data

seed	LRAP	AUPR	Wrong (top-3)
1	61.8%	56.3%	11/39
2	58.2%	42.6%	14/39
3	53.5%	50.0%	14/39
4	57.7%	57.7%	14/39
Mean	57.8%	51.7%	13.2/39
Std dev	3.4%	6.9%	1.5

Finding: AUPR swings 15 points across seeds (42.6%-57.7%). Seed 1’s 56.3% was closer to a best case than a typical case. The honest, representative number for this config is the mean (~51.7% AUPR) – nearly identical to the unfixed baseline (51.8%). Uniform loss = 0.1 does NOT reliably beat doing nothing; the earlier apparent improvement was largely a favorable split.

Collapse check across all 4 seeds (persist/privesc bias, computed directly from each seed’s saved JSON, not table-parsing):

seed	top-1 = persist/privesc	correct when guessed	still guessed when NEITHER is true
1	25/39	12/25 (48%)	11/23 (48%)
2	23/39	12/23 (52%)	10/25 (40%)
3	25/39	10/25 (40%)	15/27 (56%)
4	18/39	6/18 (33%)	12/29 (41%)

Confirmed: not a seed-1 artifact. Every seed shows roughly half of all top-1 predictions defaulting to persistence/privilege_escalation, wrong about half the time, including on 40-56% of samples where neither tactic is even remotely true. Uniform loss = 0.1 does not fix this in any of the 4 seeds.

Pipeline rebuild: EVENT__ prefix stripped, no event removal, docstrings/comments stripped

Between the sections above and below, `generalize_ip.py` (renamed `generalize_ip_and_file_path.py`) was changed: 1. `EVENT_VERB` dict (mapped raw syscalls to hand-picked English phrases, e.g. `EVENT_ANOM_ABEND` -> “abnormal termination”) replaced with a simple `EVENT_` prefix strip (`EVENT_ANOM_ABEND` -> `ANOM_ABEND`), applied uniformly to every `EVENT_*` type instead of only the ~58 explicitly mapped before. 2. `REMOVE_EVENTS` (background/administrative event types silently dropped, e.g. `AVC_STATUS`, `CRED_ACQ`, `BPF_LOAD`) removed entirely – no events are filtered out anymore, all kept. 3. `generalize_ip_and_file_path.py`’s `__main__` now auto-chains into `build_sequences.main()`, so one command rebuilds both. 4. Full pipeline re-run: 393 sequences, 0 empty. 5. `--min-events` and `--stratified` CLI flags removed from `train/test_camlds_matcher.py` – `min-events=3` and `stratified` split are now the hardcoded defaults (no flags needed). 6. All 26 `.py` files in `CAM-LDS/scripts/` had every `#` comment and `"""docstring"""` stripped (verified via `tokenize+ast`, not `regex`; every file re-verified to compile, actively-used ones functionally smoke-tested). 7. `build_sequences.py` and `train/test_camlds_matcher.py` gained `--sequences-dir` overrides, so a “without generalization” (raw graphs/, via `--prefix graph_`) variant could be built into `sequences_raw/` without overwriting the generalized `sequences/` folder. Verified byte-identical sequences/ files before/after building the raw variant.

3-way template comparison on rebuilt data (seed 1)

template	LRAP	AUPR	Wrong (top-3)	persist top-1	correct when guessed
dc (default)	42.3%	42.5%	15/30	22/30 (73%)	4/22 (18%)
b	41.0%	46.7%	17/30	22/30 (73%)	4/22 (18%)
c	47.5%	49.4%	12/30	21/30 (70%)	4/21 (19%)

The collapse is essentially identical across all 3 templates (70-73%, 18-19% correct when guessed) – template choice has no measurable effect on it. Template c wins on aggregate metrics purely from scoring the non-collapsed samples somewhat better. **Decision: use template c going forward.**

Template c, seeds 1/2/3, rebuilt (all-events-kept) sequences

seed	Train	Test	LRAP	AUPR	Wrong (top-3)	top-1 prediction distribution (how many times each tactic was predicted #1)	% on the single most-predicted tactic	correct when guessed
1	privesc 44persistence 43execution 30stealth 22discovery 21c2c 18cred_access 15init_access 14lat_mov 11collection 8def_imp 7exfil 2impact 2recon 1(total 122)	stealth 7cred_access 6discovery 6collection 5c2c 5init_access 5persistence 5execution 4privesc 4recon 2def_imp 1exfil 1impact 1lat_mov 1(total 30 samples; label counts sum to more since multi-label)	47.5%	49.4%	12/30	persistence 21cred_access 3exfil 2lat_mov 2impact 1def_imp 1	70% (persistence)	19%
2	privesc 40persistence 39execution 27c2c 20stealth 22discovery 18cred_access 16init_access 15collection 12lat_mov 10def_imp 7exfil 2impact 2recon 2(total 122)	discovery 9persistence 9privesc 8execution 7stealth 7cred_access 5init_access 4c2c 3lat_mov 2collection 1def_imp 1exfil 1impact 1recon 1(total 30 samples; label counts sum to more since multi-label)	44.8%	35.9%	13/30	persistence 14cred_access 9lat_mov 4collection 1def_imp 1exfil 1	47% (persistence)	43%

seed	Train	Test	LRAP	AUPR	Wrong (top-3)	top-1 prediction distribution (how many times each tactic was predicted #1)	% on the single most-predicted tactic	correct when guessed
3	privesc 39persistence 37execution 29stealth 24c2c 22discovery 21init_access 18cred_access 16collection 11lat_mov 9def_imp 5exfil 2impact 2recon 2(total 122)	persistence 11privesc 9discovery 6cred_access 5execution 5stealth 5def_imp 3lat_mov 3collection 2c2c 1exfil 1impact 1init_access 1recon 1(total 30 samples; label counts sum to more since multi-label)	49.3%	34.5%	10/30	privesc 17exfil 7persistence 2recon 2cred_access 1c2c 1	57% (privi- lege_escalation)	47%
Mean	—	—	47.2%	39.9%	11.7/30	—	—	—

(Per-tactic train/test breakdown is embedded directly in the Train/Test columns of the table above – no separate table needed.)

Note: reconnaissance has only 1 train example in seed 1 (below the usual stratified-split floor of 1 test + at least 1 train) – worth flagging as an even more extreme scarcity case than the general 2-per-tactic minimum seen for exfiltration/impact elsewhere. Also notable: persistence and privilege_escalation both total 48 examples each across the whole dataset – by far the largest of any tactic (next highest is execution at 34) – this is the concrete count underlying the collapse discussed throughout this file.

Two findings: 1. **The collapse is real in all 3 seeds but varies in severity (47-70%)** – seed 1 happened to be the worst case (matches the earlier lesson that single-seed numbers can mislead on magnitude, though every seed shows some real collapse). 2. **Mean AUPR on rebuilt data (39.9%) is clearly worse than mean AUPR on the original pre-rebuild data (51.7%, from the 4-seed check above).** This confirms, across multiple seeds (not just seed 1), that keeping all events (removing the old REMOVE_EVENTS noise filter) made things worse overall, not better.

Root-cause finding: extreme event repetition drowning out real signal

Quantified dataset-wide: consecutive identical repeated events make up 31.7% of all events across the dataset (14,925 of 47,041), affecting 123 of 195 steps (63%). One extreme concrete example investigated in full: **6_plugin-32** (true = collection + credential_access, T1056-001 Input Capture) – of 93 total events, 88 (95%) are just two repeated noise patterns (**apache2 setgid/setuid** alternating x31, and **apache2 connect_fail** x26). The actual attack evidence – **apache2 -> dash -> perl** (a web app spawning a shell to run a script, i.e. the real keylogger/input-capture chain) – is only 4 of 93 events, completely buried. This directly explains why the model misreads this and similar samples: mean-pooling drowns the real signal under repetition noise. NOT YET IMPLEMENTED as a fix (consecutive-event deduplication was scoped but not built).

Conclusion so far

Every attempt to actually fix the persistence/privilege_escalation collapse via the loss function (configs 4, 5) made the headline numbers worse, because the fix just relocates the reflexive-guessing behavior to different tactics rather than producing genuine discrimination. Multi-seed validation shows even the best-looking single-seed “improvement” (config #2, seed 1) doesn’t hold up on average. The pipeline rebuild (keeping all events) made the collapse and overall AUPR worse across multiple seeds, not better.

This points to two distinct, real problems, both evidenced concretely: 1. **Data scarcity** for several tactics (2-3 real training examples each) – not fixable through loss-function engineering at this dataset size. 2. **Repetition noise drowning real signal** in mean-pooled embeddings – concretely quantified (31.7% of all events dataset-wide) and demonstrated in the 6_plugin-32 case, but not yet fixed.

Recommendation: (a) treat persistence/privilege_escalation predictions as known to carry a residual bias in any write-up, (b) prioritize the consecutive-event-deduplication fix next, since it’s the most concretely evidenced unimplemented fix candidate, (c) continue averaging metrics across multiple seeds rather than trusting single-seed numbers.

Pending

- “Without generalization” (raw, ungeneralized) comparison: `sequences_raw/` built and verified (393 sequences, 0 empty, confirmed genuinely raw content e.g. real IPs and `event_execve` instead of cleaned text). Training commands for seeds 1/2/3 given to user, not yet run/reported back as of this entry.